MODULE 30 · DOMAIN 7 - DEVELOPER PRODUCTIVITY AND ENABLEMENT

Claude Code for Teams

Five settings scopes and their precedence, what goes where, least-privilege permissions, secrets, permission modes and auto mode, CLAUDE.md, subagents, Skills and plugins, and MCP for teams.

DOMAIN 7 of 7 EXAM WEIGHT 7% MODULE 30 of 38

admin@sthithapragna.com

Domain 7 at a glance

OBJECTIVE	MODULE
Configure Claude tools for teams, such as Claude Code	30
AI-assisted developer workflows	31
Debugging and operational issue resolution	31

ON THE EXAM

Seven percent of the exam, but practice material over-weights it. Learn the scopes cold, then move on.

The law of the land, the head teacher's instruction today, the note on your desk, the class rules on the wall, your own habits. **Higher always wins.**

Five scopes, highest first

SCOPE	FILE	AFFECTS
Managed	managed-settings.json, MDM, or the claude.ai console	The whole organisation
Command line	claude --settings, and flags	This session
Local	Project .claude/settings.local.json, not committed	You, in this project
Project	Project .claude/settings.json, committed	Everyone in the repository
User	~/.claude/settings.json	You, in every project

THE TIEBREAKER

Managed, command line, local, project, user. A key set higher overrides the same key set lower.

How the scopes combine

- **Same key, different scopes:** the higher scope wins.
- **Lists merge:** permission allow lists from every scope combine.
- **A deny from any scope** beats an allow from any scope.
- **Nothing overrides a managed permission rule,** not even the command line.
- **Committed project allow rules** wait until the folder is trusted.
- **/status** shows which file each active setting came from.

> ON THE EXAM
>
> Precedence decides single values. Deny-first decides permissions, across every scope.

What goes where

YOU WANT	PUT IT IN
Shared MCP servers for the repository	Project: committed .mcp.json
Team permission rules and hooks	Project: .claude/settings.json
Personal preferences, everywhere	User: ~/.claude/settings.json
Trying a setting before sharing it	Local: .claude/settings.local.json
Controls nobody may override	Managed settings
A change for one session only	Command line

THE TIEBREAKER

Shared goes in project. Personal goes in user. Non-negotiable goes in managed.

Permission rules: deny, then ask, then allow

- **Evaluated in order:** deny, then ask, then allow. First match wins.
- **Specificity does not change the order:** an allow cannot carve an exception out of a deny.
- **Syntax:** Bash(npm run *), Read(./.env), mcp__github__create_issue.
- **Deny and ask** also match commands nested inside subshells and loops.
- **A .claudeignore file has no effect.** Protect secrets with Read deny rules.

> ON THE EXAM
>
> If something is denied that you allowed, look for a deny or ask rule in any scope.

Quirk 9: Bash for all

FIRST ROLLOUT

.claude/settings.json

```
"permissions": {
  "allow": ["Bash"]
}
```

Every command, in every repository, without a prompt.

LEAST PRIVILEGE

.claude/settings.json

```
"allow": ["Bash(npm run test *)",
  "Bash(npm run lint *)",
  "Bash(git diff *)", "Bash(git log *)"],
"ask":   ["Bash(git push *)"],
"deny":  ["Bash(curl *)",
  "Read(./.env)", "Read(./secrets/**)"]
```

Narrow allows, a checkpoint on push, explicit denies.

THE TIEBREAKER

Allow what the team actually runs. Ask where a human should check. Deny what must never happen.

Wildcards: narrow means narrow

- **Put the * after the subcommand:** Bash(git log *) allows only git log.
- **Bash(git *)** allows every git command, including push and reset.
- **A * before the subcommand,** as in Bash(git * main), draws a startup warning.
- **A trailing * also matches the bare command:** Bash(ls *) matches ls.
- **Exact rules are exact:** Bash(npm run build) does not match build --watch.

> ON THE EXAM
>
> The words before the first * are what limit a rule. Make them say exactly what you mean.

Quirk 8: the committed key

COMMITTED

.claude/settings.json

```
"env": {
  "ANTHROPIC_API_KEY": "sk-ant-..."
}
```

In every clone, and in history, forever.

FIXED

After rotation

```
1. Revoke and rotate the key now.
2. Remove it, and purge history.
3. Credentials from a secret store,
   via apiKeyHelper or env vars.
4. .mcp.json uses ${GITHUB_TOKEN}.
5. Secret scanning on every push.
```

Rotate first. Deleting the line does not un-leak it.

THE TIEBREAKER

A committed secret is compromised. Rotate it first, then fix how secrets reach the tools.

Permission modes

MODE	RUNS WITHOUT ASKING	USE FOR
`default`	Reads only (shown as Manual)	Sensitive work
`acceptEdits`	Reads, edits, common file commands	Iterating on reviewed code
`plan`	Reads and exploration	Exploring before changing
`auto`	Everything, with a classifier reviewing	Long tasks, less prompt fatigue
`dontAsk`	Pre-approved tools only; the rest denied	Locked-down CI and scripts
`bypassPermissions`	Everything	Isolated containers only

THE TIEBREAKER

Deny rules block in every mode. Managed settings can disable bypass and auto mode.

Auto mode: what the classifier blocks by default

- **Downloading and running code,** such as curl piped to bash.
- **Sending sensitive data** to external endpoints.
- **Production deploys and migrations,** force push, granting permissions.
- **Merging a pull request no human approved,** or disabling CI checks.
- **Add ask rules** for your own checkpoints; stated boundaries such as "don't push" also block.

> ON THE EXAM
>
> Auto mode reduces prompts, not responsibility. Ask and deny rules still define your boundaries.

CLAUDE.md and rules: shared conventions

- **Project CLAUDE.md, committed:** build commands, style, architecture notes, review norms.
- **Loaded from managed, user, project and local** locations, and combined.
- **.claude/rules/** with path patterns, for conventions that apply to part of the repo.
- **Keep it short:** it costs context in every session; /context shows usage.
- **Instructions guide; permissions enforce.** Never rely on CLAUDE.md to block an action.

> ON THE EXAM
>
> Conventions go in CLAUDE.md. Controls go in permissions and hooks.

Subagents, Skills, hooks, plugins: which for what

MECHANISM	USE IT FOR
Subagent (.claude/agents)	A recurring specialised task, with its own context, tools and model
Skill (.claude/skills)	A procedure or knowledge loaded only when relevant
Hook	Deterministic enforcement at an event; exit code 2 blocks
Plugin and marketplace	Distributing bundles of all of these across teams
MCP server	Access to an external system or data source

THE TIEBREAKER

Isolate work with a subagent, teach a procedure with a Skill, enforce with a hook, distribute with a plugin.

Federated Skills, with a central gate

01
Author
Teams write Skills where the knowledge lives.

02
Propose
Pull request to the shared repository.

03
Review
Security, quality, overlap, an owner.

04
Publish
To the organisation's marketplace.

05
Restrict
Managed settings allow known marketplaces only.

06
Install
Teams enable what they need.

ON THE EXAM

Authorship is federated. Approval is central. Distribution is controlled.

MCP for teams

- **Shared servers in the committed .mcp.json;** each developer approves them once.
- **Secrets by variable expansion:** ${VAR}, never a literal token.
- **Tool search is on by default** and defers MCP tools, keeping context small.
- **Managed allow and deny lists** for servers; a deny always wins.
- **/mcp and claude mcp list** show what is connected, and from where.

> ON THE EXAM
>
> Many MCP servers no longer means a full context window. Tool search loads tools when they are needed.

> Tallyhawk wants every engineer working in the monorepo to get the same help-centre and GitHub MCP servers automatically.

Where should Corvane configure them?

A. In the project's committed .mcp.json, with credentials supplied through variable expansion

B. In each engineer's user settings, so everyone configures and maintains their own copy

C. In each engineer's local settings file for the repository, kept out of version control

D. In a CLAUDE.md note asking engineers to add both servers when they first clone

◆ PAUSE AND DECIDE

Where should Corvane configure them?

A. In the project's committed .mcp.json, with credentials supplied through variable expansion

Shared, committed, no secrets in the file.

B. In each engineer's user settings, so everyone configures and maintains their own copy

Personal scope for a shared need.

C. In each engineer's local settings file for the repository, kept out of version control

Not committed, so not shared.

D. In a CLAUDE.md note asking engineers to add both servers when they first clone

Asks instead of configuring.

THE SIGNAL

Shared MCP servers for a repository belong in project scope, with secrets by variable.

> A Tallyhawk engineer wants her preferred model and notification settings in every repository she works in.
>
> They are personal choices, not team standards.

Where should she put them?

A. In each repository's committed project settings

B. In her user settings file in her home folder

C. In managed settings, so they are always applied

D. In each repository's CLAUDE.md file

◆ PAUSE AND DECIDE

Where should she put them?

A In each repository's committed project settings

Imposes her choices on the team.

B In her user settings file in her home folder

Personal, and applies everywhere.

C In managed settings, so they are always applied

Organisation-wide control, for a preference.

D In each repository's CLAUDE.md file

Instructions, not settings.

THE SIGNAL

Personal preferences that follow you everywhere go in user scope.

> Tallyhawk's security team requires that no developer, in any repository, can let Claude read .env files or use bypass mode.

How should Corvane enforce it?

A. Add deny rules to each repository's project settings, and remind teams to keep them

B. Add a line to every CLAUDE.md saying never read .env files or use bypass mode

C. Managed settings with Read deny rules and bypass mode disabled

D. Email developers a user settings file with the deny rules, and ask them to install it

◆ PAUSE AND DECIDE

How should Corvane enforce it?

A Add deny rules to each repository's project settings, and remind teams to keep them

Editable, and new repositories miss it.

B Add a line to every CLAUDE.md saying never read .env files or use bypass mode

Guidance, not enforcement.

C Managed settings with Read deny rules and bypass mode disabled

Non-overridable, everywhere.

D Email developers a user settings file with the deny rules, and ask them to install it

Optional, and easy to change.

THE SIGNAL

Non-negotiable controls go in managed settings, which nothing can override.

> A Tallyhawk engineer committed an API key inside the project's .claude/settings.json. It was pushed an hour ago.

What should happen first?

A Delete the line from the settings file, and commit the fix straight away

B Make the repository private first, then remove the key in a later commit

C Move the key into the local settings file instead, which is never committed

D Revoke and rotate the key, then remove it from the repository and its history

◆ PAUSE AND DECIDE

What should happen first?

A. Delete the line from the settings file, and commit the fix straight away
The key stays in history.

B. Make the repository private first, then remove the key in a later commit
Already exposed to every clone.

C. Move the key into the local settings file instead, which is never committed
Still a literal secret on disk.

D. Revoke and rotate the key, then remove it from the repository and its history
Rotate first; it is compromised.

THE SIGNAL

A committed secret is compromised. Rotate first, then remove, then fix how secrets are supplied.

> Tallyhawk's first Claude Code rollout allows the whole Bash tool in every repository.
>
> Teams mainly run tests, linting and read-only git commands.

What should Corvane recommend?

A. Narrow allow rules for those commands, ask before git push, and deny downloads and secret reads

B. Remove the Bash tool entirely, so Claude cannot run any command at all

C. Switch every developer to bypass mode inside containers, so prompts disappear

D. Keep the Bash allow as it is, and rely on auto mode's classifier to catch problems

◆ PAUSE AND DECIDE

What should Corvane recommend?

A Narrow allow rules for those commands, ask before git push, and deny downloads and secret reads

Least privilege, with checkpoints.

B Remove the Bash tool entirely, so Claude cannot run any command at all

Over-corrects; loses the value.

C Switch every developer to bypass mode inside containers, so prompts disappear

Removes every check; containers only.

D Keep the Bash allow as it is, and rely on auto mode's classifier to catch problems

A blanket allow stays; a classifier is not policy.

THE SIGNAL

Least privilege: allow what is needed, ask where a person should check, deny what must never happen.

> A repository's project settings allow Bash(npm run test *). An engineer's test command is still blocked.
>
> The project file has no deny rules.

What is the most likely cause?

A The allow rule is not specific enough, and needs the full command

B A matching deny rule in another scope, such as managed or user settings

C Project settings never apply to Bash commands

D Claude Code must be reinstalled after any settings change

◆ PAUSE AND DECIDE

What is the most likely cause?

A The allow rule is not specific enough, and needs the full command

Specificity never overrides a deny.

B A matching deny rule in another scope, such as managed or user settings

Deny from any scope wins; check status.

C Project settings never apply to Bash commands

False: project rules apply.

D Claude Code must be reinstalled after any settings change

Settings reload without a restart.

THE SIGNAL

When an allowed action is blocked, look for a deny or ask rule in another scope. Deny wins everywhere.

> Tallyhawk's security reviewers run the same injection, secrets and permission review on every pull request.
>
> Pasting the checklist each time fills the main conversation.

What should Corvane set up?

A. Paste a shorter version of the checklist into each conversation as it starts

B. Add the full checklist to the project CLAUDE.md, so it is always available

C. A committed security-review subagent with its own instructions, tools and context

D. A hook that prints the checklist into the conversation before every tool call

◆ PAUSE AND DECIDE

What should Corvane set up?

A Paste a shorter version of the checklist into each conversation as it starts

Still manual, still in the main context.

B Add the full checklist to the project CLAUDE.md, so it is always available

Costs context in every session, for everyone.

C A committed security-review subagent with its own instructions, tools and context

Recurring, specialised, isolated.

D A hook that prints the checklist into the conversation before every tool call

A hook enforces; it does not review.

THE SIGNAL

Recurring specialised work belongs in a subagent with its own context.

> Tallyhawk's eighteen teams want to share Claude Code Skills.
>
> Security insists that nothing unreviewed reaches developers.

Which two practices should Corvane recommend? Choose two.

A. Teams author Skills and propose them by pull request to a shared repository

B. A central review for security, quality and overlap before publishing to the organisation's marketplace

C. Each team copies the Skills it likes into its own repository

D. Allow any public marketplace, so teams can find the best Skills

E. Ban team-authored Skills, and have one central team write them all

◆ PAUSE AND DECIDE

Which two practices should Corvane recommend? Choose two.

A. Teams author Skills and propose them by pull request to a shared repository
Federated authorship.

B. A central review for security, quality and overlap before publishing to the organisation's marketplace
Central gate before distribution.

C. Each team copies the Skills it likes into its own repository
Copies drift, unreviewed.

D. Allow any public marketplace, so teams can find the best Skills
Unreviewed sources.

E. Ban team-authored Skills, and have one central team write them all
A bottleneck that loses team knowledge.

THE SIGNAL

Federated authorship, a central gate, controlled distribution.

In five lines

- **Managed, command line, local, project, user;** higher wins, lists merge.
- **Shared in project, personal in user,** non-negotiable in managed.
- **Deny, then ask, then allow;** a deny in any scope wins; narrow wildcards.
- **Never commit secrets;** rotate first, then use variables and helpers.
- **CLAUDE.md guides; permissions and hooks enforce;** subagents isolate.

COMING UP

Next: AI-assisted workflows, rollout and troubleshooting.